REVISION CHEAT SHEET:

Inheritance in C#

INHERITANCE

Interview-ready notes · examples · C# · diagrams

Topic 13 of 40

Inheritance is one of the core ideas in object-oriented programming.

It allows one class to:

EXAMPLE / FLOW

```
Reuse
+
Extend
+
Specialize
```

another class.

For example, imagine a fundraising system containing a general concept:

EXAMPLE / FLOW

```
FinancialTransaction
```

and more specific transaction types:

EXAMPLE / FLOW

```
Gift

Refund

Adjustment
```

Instead of repeating common members such as:

EXAMPLE / FLOW

```
TransactionId
TenantId
Amount
CreatedAtUtc
```

in every class, we may place common state and behavior in a base class:

EXAMPLE / FLOW

```
FinancialTransaction
```

and derive more specialized types from it.

The core relationship is:

EXAMPLE / FLOW

```
Base Class
    →
General Concept

Derived Class
```

```
    →
More Specialized Concept
```

INHERITANCE

For example:

EXAMPLE / FLOW

```
FinancialTransaction
        ↓
       Gift
```

Meaning:

EXAMPLE / FLOW

```
A Gift IS A FinancialTransaction
```

That phrase—**"is a"**—is extremely important when deciding whether inheritance is appropriate.

Microsoft defines inheritance as the ability to create classes that reuse, extend, and modify behavior defined by another class. C# supports a single direct base class for each class, while inheritance itself is transitive through a hierarchy.

Topic Coverage

This topic covers:

- inheritance definition;
- base classes;
- derived classes;
- parent/child terminology;
- inheritance syntax;
- `is-a` relationship;
- single inheritance;
- inheritance hierarchies;
- transitive inheritance;
- `System.Object`;
- inherited members;
- inaccessible private members;
- `protected`;
- constructors and inheritance;
- `base`;
- constructor chaining through inheritance;
- member extension;
- method reuse;
- virtual methods;
- `override`;
- runtime dispatch introduction;
- virtual properties;
- `base.Method()`;
- method hiding;
- `new`;
- `new` vs `override`;
- `sealed` classes;
- `sealed override`;

- abstract-class preview;
- upcasting;
- downcasting preview;
- `is` pattern checks;
- inheritance vs composition;
- fragile base-class problems;
- when to use inheritance;
- when not to use inheritance;
- ASP.NET Core examples;
- Fundraising Platform examples;
- performance considerations;
- security considerations;
- practical exercises;
- interview questions.

INHERITANCE

The most important mental model is:

EXAMPLE / FLOW

```
General Type
    →
Base Class
    →
Common State + Common Behavior
    →
Derived Class
    →
Specialized State + Specialized Behavior
```

What Is Inheritance?

Definition

Inheritance allows one class to derive from another class and automatically become part of that type hierarchy.

Example:

C#

```
public class FinancialTransaction
{
}
```

Derived type:

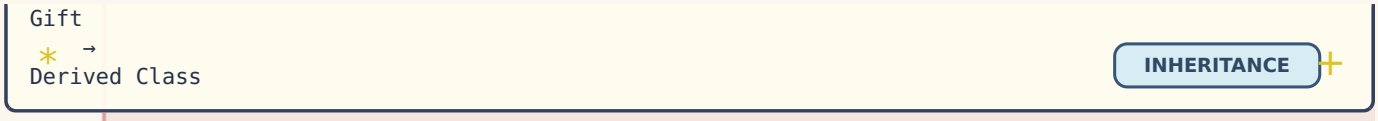
C#

```
public class Gift :
    FinancialTransaction
{
}
```

Now:

EXAMPLE / FLOW

```
FinancialTransaction
    →
Base Class
```



`Gift` inherits members from `FinancialTransaction`, subject to accessibility rules.

Base Class

Definition

A **base class** contains members that derived classes can inherit and potentially specialize.

Example:

```C#
public class FinancialTransaction
{
    public long TransactionId
    {
        get;
        init;
    }

    public decimal Amount
    {
        get;
        protected set;
    }
}
```

This class contains state common to financial transactions.

Derived Class

Definition

A **derived class** inherits from another class.

Example:

```C#
public class Gift :
    FinancialTransaction
{
    public long ConstituentId
    {
        get;
        init;
    }
}
```

The derived class has access to its own members:

EXAMPLE / FLOW

```
ConstituentId
```

and inherited members:

EXAMPLE / FLOW

```
TransactionId
Amount
```

according to accessibility rules.

Parent and Child Terminology

You may hear:

```
EXAMPLE / FLOW
Parent Class
Child Class
```

These usually mean:

```
EXAMPLE / FLOW
Parent
    →
Base Class

Child
    →
Derived Class
```

In professional C# documentation, these terms are more common:

```
EXAMPLE / FLOW
Base Class

Derived Class
```

Basic Inheritance Syntax

Syntax:

```
C#
public class DerivedClass :
    BaseClass
{
}
```

Example:

```
C#
public class Gift :
    FinancialTransaction
{
}
```

The colon:

```
EXAMPLE / FLOW
:
```

indicates inheritance.

Simple Example

```
C#
public class FinancialTransaction
{
```

```
    public decimal Amount
    {
        get;
        protected set;
    }

    public DateTime CreatedAtUtc
    {
        get;
        init;
    }
}

public class Gift :
    FinancialTransaction
{
    public long ConstituentId
    {
        get;
        init;
    }
}
```

Create:

```
C#
Gift gift =
    new Gift
    {
        ConstituentId = 5001,

        CreatedAtUtc =
            DateTime.UtcNow
    };
```

The `Gift` type contains inherited capabilities from:

```
EXAMPLE / FLOW
FinancialTransaction
```

plus its own additional members.

The "Is-A" Relationship

Inheritance should normally represent an:

```
EXAMPLE / FLOW
IS-A
```

relationship.

Examples:

```
EXAMPLE / FLOW
Gift
IS A
FinancialTransaction
```

EXAMPLE / FLOW

```
RecurringGift
IS A
Gift
```

EXAMPLE / FLOW

```
RefundTransaction
IS A
FinancialTransaction
```

This relationship is fundamental.

Good Inheritance Relationship

EXAMPLE / FLOW

```
Payment
    →
CreditCardPayment

CreditCardPayment
IS A
Payment
```

Reasonable.

Bad Inheritance Relationship

EXAMPLE / FLOW

```
Gift
    →
DatabaseConnection
```

A gift is **not** a database connection.

So:

C#

```
class Gift :
    DatabaseConnection
```

would be conceptually wrong even if the compiler technically allowed the types involved.

"Has-A" Usually Suggests Composition

Suppose:

EXAMPLE / FLOW

```
Gift
HAS A
Campaign
```

That does not mean:

EXAMPLE / FLOW

```
Gift
IS A
Campaign
```

So inheritance is inappropriate.

Instead:


```C#
public class Gift
{
    public Campaign? Campaign
    {
        get;
        init;
    }
}
```

This is composition/association.

We will study this deeply in Topic 16.

Single Class Inheritance

C# supports:

EXAMPLE / FLOW

```
one direct base class
```

for a class.

Example:

```C#
public class Gift :
    FinancialTransaction
{
}
```

is valid.

But this is not:

```C#
public class Gift :
    FinancialTransaction,
    AuditableEntity
{
}
```

if both are classes.

C# does not support multiple direct class inheritance. A class can derive from one class while implementing multiple interfaces.

Why Only One Base Class?

Multiple class inheritance can create ambiguity.

Imagine:

EXAMPLE / FLOW

```
Base A
    →
Process()
```

```
Base B
 *  →
Process()

Derived C
    →
inherits both
```

Which:

EXAMPLE / FLOW

```
Process()
```

should `C` receive?

Languages supporting multiple inheritance need additional rules to resolve such situations.

C# avoids this class-level ambiguity by allowing:

EXAMPLE / FLOW

```
One base class
+
Multiple interfaces
```

Interfaces will be covered in Topic 15.

Inheritance Is Transitive

Suppose:

C#

```
public class Transaction
{
}

public class Donation :
    Transaction
{
}

public class Gift :
    Donation
{
}
```

Then:

EXAMPLE / FLOW

```
Gift
IS A
Donation
```

and because inheritance is transitive:

EXAMPLE / FLOW

```
Gift
IS ALSO A
Transaction
```

Microsoft explicitly defines inheritance as transitive.

Inheritance Hierarchy

Example:

EXAMPLE / FLOW

```
System.Object
      ↓
FinancialTransaction
      ↓
Donation
      ↓
Gift
      ↓
RecurringGift
```

This is called an:

EXAMPLE / FLOW

```
Inheritance Hierarchy
```

Each level becomes more specialized.

Direct vs Indirect Base Class

Given:

EXAMPLE / FLOW

```
Transaction
    ↓
Donation
    ↓
Gift
```

For `Gift`:

EXAMPLE / FLOW

```
Donation
```

is the:

EXAMPLE / FLOW

```
Direct Base Class
```

`Transaction` is an:

EXAMPLE / FLOW

```
Indirect Base Class
```

System.Object

Almost every normal C# class ultimately derives from:

EXAMPLE / FLOW

```
System.Object
```

even if you do not explicitly write it.

Example:

```C#
public class Gift
{
}
```

conceptually participates in:

```EXAMPLE / FLOW
System.Object
    ↓
Gift
```

That is why every normal class gets members such as:

```EXAMPLE / FLOW
ToString()

Equals()

GetHashCode()

GetType()
```

through the base `object` type. Microsoft notes that ordinary class hierarchies ultimately derive from `System.Object`.

Explicit Object Inheritance Is Usually Unnecessary

This:

```C#
public class Gift :
    object
{
}
```

is legal.

But normally just write:

```C#
public class Gift
{
}
```

because:

```EXAMPLE / FLOW
object
```

is implicit.

What Members Are Inherited?

A derived class inherits base-class members such as:

```EXAMPLE / FLOW
Fields
```

```
Methods
*
Properties
Events
Indexers
Nested Types
Constants
```

INHERITANCE

with accessibility determining whether those inherited members can be directly accessed.

However, the following are not inherited:

EXAMPLE / FLOW

```
Instance Constructors

Static Constructors

Finalizers
```

This distinction is explicitly defined in the C# language specification.

Constructors Are Not Inherited

Suppose:

C#

```
public class Transaction
{
    public Transaction(
        decimal amount)
    {
    }
}
```

Then:

C#

```
public class Gift :
    Transaction
{
}
```

does not inherit:

EXAMPLE / FLOW

```
Transaction(decimal)
```

as though it were:

EXAMPLE / FLOW

```
Gift(decimal)
```

The derived type must define its own constructors.

Derived Constructor Can Call Base Constructor

INHERITANCE

Example:

C#

```
public class Transaction
{
    public Transaction(
        decimal amount)
    {
        Amount = amount;
    }

    public decimal Amount
    {
        get;
    }
}

public class Gift :
    Transaction
{
    public Gift(
        decimal amount,
        long constituentId)
        : base(
            amount)
    {
        ConstituentId = constituentId;
    }

    public long ConstituentId
    {
        get;
    }
}
```

Construction:

C#

```
Gift gift =
    new Gift(
        amount: 1000m,
        constituentId: 5001);
```

Flow:

EXAMPLE / FLOW

```
Gift Constructor Selected
    →
base(amount)
    →
Transaction Constructor
    →
Transaction State Initialized
    →
Gift Constructor Body
    →
Gift State Initialized
```

The `base(...)` constructor initializer selects which base-class constructor runs.

Base Constructor Runs Before Derived Constructor Body

This is extremely important.

Given:

```
EXAMPLE / FLOW
Base
    ↓
Derived
```

construction runs conceptually:

```
EXAMPLE / FLOW
Base Initialization
    →
Derived Initialization
```

Why?

Because before the derived class can safely work with its inherited base portion, that base portion should be initialized.

Example

```
C#
public class Transaction
{
    public Transaction()
    {
        Console.WriteLine(
            "Transaction constructor");
    }
}

public class Gift :
    Transaction
{
    public Gift()
    {
        Console.WriteLine(
            "Gift constructor");
    }
}
```

Usage:

```
C#
Gift gift = new Gift();
```

Output conceptually:

```
EXAMPLE / FLOW
Transaction constructor

Gift constructor
```

Implicit base()

If you write:

```C#
public class Gift :
    Transaction
{
    public Gift()
    {
    }
}
```

and `Transaction` has an accessible parameterless constructor, the compiler effectively calls:

```C#
base()
```

automatically.

Explicit base(...)

If the base class has no accessible parameterless constructor:

```C#
public class Transaction
{
    public Transaction(
        decimal amount)
    {
    }
}
```

then derived class must call a valid base constructor:

```C#
public class Gift :
    Transaction
{
    public Gift(
        decimal amount)
        : base(
            amount)
    {
    }
}
```

The base Keyword

`base` lets a derived class access members from its direct base class.

Two common uses:

EXAMPLE / FLOW

```
Call Base Constructor

Call Base Member Implementation
```

Microsoft documents these as the major uses of `base`.

Calling a Base Method

Example:

```C#
public class Transaction
{
    public virtual void PrintSummary()
    {
        Console.WriteLine(
            "Transaction");
    }
}

public class Gift :
    Transaction
{
    public override void PrintSummary()
    {
        base.PrintSummary();

        Console.WriteLine(
            "Gift");
    }
}
```

Output:

EXAMPLE / FLOW

```
Transaction
Gift
```

The derived method:

```C#
base.PrintSummary();
```

explicitly invokes the base-class implementation.

base Is Not an Object Variable

You should think of:

EXAMPLE / FLOW

```
base
```

as a way of referring to the inherited base implementation.

It is not simply another independent object.

The derived instance contains its base-class portion.

Accessibility in Inheritance

The most relevant access modifiers are:

EXAMPLE / FLOW

```
public

protected
```

```
private

internal

protected internal

private protected
```

Inheritance does not automatically make every base member directly accessible.

public Member

```C#
public class Transaction
{
    public decimal Amount
    {
        get;
        set;
    }
}
```

Derived class can access:

```C#
Amount
```

because it is public.

Any other appropriate caller can also access it.

protected Member

Example:

```C#
public class Transaction
{
    protected decimal Amount
    {
        get;
        set;
    }
}

public class Gift :
    Transaction
{
    public void Increase(
        decimal amount)
    {
        Amount += amount;
    }
}
```

`Amount` is available inside:

EXAMPLE / FLOW

```
Transaction
+
Derived Types
```

INHERITANCE

Microsoft defines `protected` as accessible within the declaring class and derived classes.

Why protected Exists

`private` may be too restrictive when a base type intentionally exposes extension state/behavior to derived types.

`public` may be too open because every caller would have access.

`protected` provides an inheritance-specific visibility level.

Use protected Carefully

Bad design:

C#

```
protected decimal Amount;

protected bool IsCancelled;

protected string Status;
```

Every derived class can modify raw state arbitrarily.

This weakens encapsulation.

Often better:

C#

```
public decimal Amount
{
    get;
    private set;
}

protected void SetAmount(
    decimal amount)
{
    if (amount <= 0m)
    {
        throw new ArgumentOutOfRangeException(
            nameof(amount));
    }

    Amount = amount;
}
```

Now derived classes use a controlled operation.

private Base Members

Example:

C#

```
public class Transaction
{
```

```
    private decimal _amount;
}
```

A normal derived class cannot directly write:

```C#
_amount
```

because the field is private.

However, the field still belongs to the base portion of the object.

The language specification distinguishes between **inheritance** and **accessibility**: members other than constructors/finalizers are inherited, but private accessibility can prevent direct use from derived source code.

Access Private Base State Through Protected/Public Members

Example:

```C#
public class Transaction
{
    private decimal _amount;

    public decimal Amount
        =>
        _amount;

    protected void ChangeAmountCore(
        decimal amount)
    {
        _amount = amount;
    }
}
```

Derived class:

```C#
public class Gift :
    Transaction
{
    public void ChangeGiftAmount(
        decimal amount)
    {
        ChangeAmountCore(
            amount);
    }
}
```

Derived class does not manipulate `_amount` directly.

Base class preserves control.

Inheritance and Encapsulation

Good inheritance does **not** mean:

EXAMPLE / FLOW

```
Derived classes get unlimited access
to base implementation details.
```

Good base-class design still uses:

EXAMPLE / FLOW

```
private
```

for implementation details.

Expose only intentional extension points through:

EXAMPLE / FLOW

```
protected
```

or:

EXAMPLE / FLOW

```
protected virtual
```

members where appropriate.

Extending a Base Class

Derived classes can add their own members.

Example:

C#

```
public class Transaction
{
    public decimal Amount
    {
        get;
        protected set;
    }
}

public class Gift :
    Transaction
{
    public long ConstituentId
    {
        get;
        init;
    }

    public long? CampaignId
    {
        get;
        init;
    }
}
```

`Gift` extends `Transaction` with:

```
ConstituentId

CampaignId
```

Reusing Base Behavior

Example:

```C#
public class Transaction
{
    public decimal Amount
    {
        get;
        protected set;
    }

    public bool HasPositiveAmount()
    {
        return Amount > 0m;
    }
}
```

Derived:

```C#
public class Gift :
    Transaction
{
}
```

`Gift` can reuse:

```C#
gift.HasPositiveAmount();
```

No need to reimplement identical logic.

The Problem with Copy-Paste Without Inheritance

Without inheritance:

```C#
public class Gift
{
    public long TransactionId { get; set; }

    public decimal Amount { get; set; }

    public DateTime CreatedAtUtc { get; set; }
}
```

and:

```C#
public class Refund
{
    public long TransactionId { get; set; }
```

```
    public decimal Amount { get; set; }

    public DateTime CreatedAtUtc { get; set; }
}
```

Potential duplication:

EXAMPLE / FLOW

```
TransactionId

Amount

CreatedAtUtc
```

A meaningful common abstraction could reduce duplication:

EXAMPLE / FLOW

```
FinancialTransaction
```

But Duplication Alone Is Not Enough Reason for Inheritance

This is extremely important.

Two classes may contain similar fields without being true:

EXAMPLE / FLOW

```
is-a
```

relationships.

Example:

EXAMPLE / FLOW

```
Gift
```

and:

EXAMPLE / FLOW

```
Invoice
```

might both contain:

EXAMPLE / FLOW

```
Amount
Date
Currency
```

That does not automatically mean one should inherit from the other.

Inheritance should model substitutability and conceptual specialization—not merely code reuse.

Virtual Methods

Definition

A `virtual` method tells C#:

Derived classes are allowed to provide a specialized implementation.

Example:

```C#
public class Transaction
{
    public virtual string GetDescription()
    {
        return "Financial transaction";
    }
}
```

Derived:

```C#
public class Gift :
    Transaction
{
    public override string GetDescription()
    {
        return "Fundraising gift";
    }
}
```

Microsoft documents `virtual` as allowing a derived type to override a method, property, indexer, or event. Methods are non-virtual by default.

Methods Are Non-Virtual by Default

This is important.

Example:

```C#
public void Process()
{
}
```

is not automatically overridable.

A derived class cannot legally write:

```C#
public override void Process()
{
}
```

unless the base member is:

EXAMPLE / FLOW

```
virtual

abstract

or already override
```

and not sealed.

Why Non-Virtual by Default?

Inheritance extension points change the contract of your class.

Once you mark something:

```
EXAMPLE / FLOW
virtual
```

you are saying:

```
EXAMPLE / FLOW
Derived classes may change this behavior.
```

That creates responsibilities for the base class author.

Therefore, virtual behavior should be intentional.

override

A derived class uses:

```
EXAMPLE / FLOW
override
```

to replace the implementation of an inherited virtual member.

Example:

```
C#
public class Transaction
{
    public virtual decimal CalculateProcessingFee()
    {
        return 0m;
    }
}

public class Gift :
    Transaction
{
    public override decimal CalculateProcessingFee()
    {
        return Amount * 0.02m;
    }

    public decimal Amount
    {
        get;
        init;
    }
}
```

Runtime Dispatch Preview

Suppose:

```
C#
Transaction transaction =
    new Gift
    {
```

```
        Amount =
            1000m
    };
```

Then:

```
C#
decimal fee = transaction.CalculateProcessingFee();
```

Even though the variable type is:

```
EXAMPLE / FLOW
Transaction
```

the actual runtime object is:

```
EXAMPLE / FLOW
Gift
```

So the runtime selects the `Gift` override.

This is runtime polymorphism and will be the main subject of Topic 14. Microsoft states that virtual dispatch chooses the implementation based on the runtime object type.

Virtual Dispatch Flow

```
EXAMPLE / FLOW
Base Reference
    →
Transaction

Runtime Object
    →
Gift

Call Virtual Method
    →
Runtime checks most-derived override
    →
Gift implementation runs
```

Example

```
C#
public class Transaction
{
    public virtual string GetTypeName()
    {
        return "Transaction";
    }
}

public class Gift :
    Transaction
{
    public override string GetTypeName()
    {
        return "Gift";
```

```
    }
}
```

Call:

```C#
Transaction transaction = new Gift();

Console.WriteLine(
    transaction.GetTypeName());
```

Output:

```
EXAMPLE / FLOW
Gift
```

because `GetTypeName` is virtual/overridden.

Override Rules

For a method override, the inherited target must be:

```
EXAMPLE / FLOW
virtual

abstract

or
override
```

and must not be:

```
EXAMPLE / FLOW
sealed
```

The overriding member must follow compatibility rules for signature, accessibility, and return type. C# supports covariant return types in appropriate reference-type scenarios.

For beginner study, remember:

```
EXAMPLE / FLOW
Base virtual member
    →
Derived override member
```

Access Level of Override

You normally cannot arbitrarily change:

```
EXAMPLE / FLOW
public virtual
```

into:

```
EXAMPLE / FLOW
protected override
```

or another unrelated accessibility.

The override contract must match the base member's declared accessibility, subject to specific cross-assembly `protected internal` rules.

Virtual Property

Properties can also be virtual.

Base:

```C#
public class Transaction
{
    public virtual string DisplayName
    {
        get
        {
            return "Transaction";
        }
    }
}
```

Derived:

```C#
public class Gift :
    Transaction
{
    public override string DisplayName
    {
        get
        {
            return "Gift";
        }
    }
}
```

Properties, indexers, and events can participate in virtual override behavior just like methods.

Virtual Auto-Property

Example:

```C#
public virtual string Name
{
    get;
    set;
} = string.Empty;
```

Derived class may override that property.

This is commonly seen in frameworks, though you should only mark properties virtual when derived customization is intentional.

Calling Base Implementation from Override

Sometimes derived behavior should:

EXAMPLE / FLOW

```
Reuse Base Behavior
+
Add Specialized Behavior
```

INHERITANCE

Example:

C#

```
public class Transaction
{
    public virtual void Validate()
    {
        if (Amount <= 0m)
        {
            throw new InvalidOperationException(
                "Amount must be positive.");
        }
    }

    public decimal Amount
    {
        get;
        protected set;
    }
}
```

Derived:

C#

```
public class Gift :
    Transaction
{
    public long ConstituentId
    {
        get;
        init;
    }

    public override void Validate()
    {
        base.Validate();

        if (ConstituentId <= 0)
        {
            throw new InvalidOperationException(
                "Constituent is required.");
        }
    }
}
```

Flow:

EXAMPLE / FLOW

```
Gift.Validate()
    →
Transaction.Validate()
    →
Base Amount Rule
    →
Gift-Specific Constituent Rule
```

When to Call base.Method()

Use it when the derived implementation should preserve base behavior.

Do not call it automatically.

Example:

C#
```
public override void Process()
{
    base.Process();

    // derived behavior
}
```

is appropriate only when:

EXAMPLE / FLOW
```
base.Process()
```

is genuinely part of the derived operation's contract.

Method Hiding

Now we reach an important distinction.

Suppose base method is not virtual:

C#
```
public class Transaction
{
    public void Print()
    {
        Console.WriteLine(
            "Transaction");
    }
}
```

Derived declares another method:

C#
```
public class Gift :
    Transaction
{
    public new void Print()
    {
        Console.WriteLine(
            "Gift");
    }
}
```

This is:

EXAMPLE / FLOW
```
Method Hiding
```

not:

EXAMPLE / FLOW

```
Method Overriding
```

INHERITANCE

Microsoft documents `new` as explicitly hiding an accessible inherited member rather than overriding it.

`new` Keyword in Inheritance

The `new` modifier says:

> I intentionally define a new member that hides a base member with the same name/signature.

Example:

C#

```
public new void Print()
{
}
```

What Happens Without `new`?

Suppose:

C#

```
public class Transaction
{
    public void Print()
    {
    }
}

public class Gift :
    Transaction
{
    public void Print()
    {
    }
}
```

C# allows the hiding behavior but generates a compiler warning because your intent is ambiguous.

Adding:

EXAMPLE / FLOW

```
new
```

makes the hiding explicit.

`override` vs `new`

This is one of the most important interview topics.

override

EXAMPLE / FLOW

```
Extends/replaces virtual behavior
```

```
Runtime object type
controls dispatch
```

new

EXAMPLE / FLOW

```
Hides base member

Compile-time reference type
strongly affects which member is selected
```

Example: override

C#

```
public class Transaction
{
    public virtual void Print()
    {
        Console.WriteLine(
            "Transaction");
    }
}

public class Gift :
    Transaction
{
    public override void Print()
    {
        Console.WriteLine(
            "Gift");
    }
}
```

Call:

C#

```
Transaction transaction = new Gift();

transaction.Print();
```

Output:

EXAMPLE / FLOW

```
Gift
```

Example: new

C#

```
public class Transaction
{
    public void Print()
    {
        Console.WriteLine(
            "Transaction");
    }
}

public class Gift :
    Transaction
```

```
{
    public new void Print()
    {
        Console.WriteLine(
            "Gift");
    }
}
```

Now:

C#
```
Transaction transaction = new Gift();

transaction.Print();
```

Output:

EXAMPLE / FLOW
```
Transaction
```

But:

C#
```
Gift gift = new Gift();

gift.Print();
```

Output:

EXAMPLE / FLOW
```
Gift
```

override Flow

EXAMPLE / FLOW
```
Reference Type:
Transaction

Runtime Type:
Gift

Virtual Method Call
    →
Gift override
```

new Flow

EXAMPLE / FLOW
```
Reference Type:
Transaction

Hidden Member Call
    →
Transaction member
```

Versus:

EXAMPLE / FLOW

```
Reference Type:
Gift

Hidden Member Call
    →
Gift member
```

Why `new` Can Be Confusing

Consider:

C#

```
Gift gift = new Gift();

gift.Print();
```

may give:

EXAMPLE / FLOW

```
Gift
```

but:

C#

```
Transaction transaction = gift;

transaction.Print();
```

may give:

EXAMPLE / FLOW

```
Transaction
```

for hidden non-virtual members.

The same object appears to behave differently depending on the reference type.

That is why method hiding should be used intentionally and relatively rarely.

Prefer override for True Specialization

If the design intends:

EXAMPLE / FLOW

```
Gift provides a specialized implementation
of Transaction behavior
```

then the base class should usually provide:

EXAMPLE / FLOW

```
virtual
```

and the derived type should use:

EXAMPLE / FLOW

```
override
```

rather than hiding with `new`.

Why Does new Exist?

It supports scenarios such as:

EXAMPLE / FLOW

```
Versioning

Intentional member hiding

Base class later introduces member
with same name
```

Microsoft specifically explains that `new` and `override` help library versions evolve while making the programmer's intention explicit.

Sealed Class

Definition

A class marked:

EXAMPLE / FLOW

```
sealed
```

cannot be inherited.

Example:

C#

```
public sealed class GiftReference
{
}
```

This is invalid:

C#

```
public class SpecialGiftReference :
    GiftReference
{
}
```

Microsoft documents `sealed` classes as preventing further derivation.

Why Seal a Class?

Possible reasons:

EXAMPLE / FLOW

```
Inheritance would break invariants

Type wasn't designed for extension

Security-sensitive implementation

Value-like/reference wrapper semantics

Prevent unsupported customization
```

Do not blindly seal everything, but also do not make a class inheritable accidentally.

Example

C#
```
public sealed class GiftReference
{
    public GiftReference(
        string value)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(
            value);

        Value = value;
    }

    public string Value
    {
        get;
    }
}
```

If this type's validity depends on strict internal rules, preventing subclass behavior may make sense.

sealed override

A derived class can override a virtual member and then prevent further derived classes from overriding that same member.

Example:

C#
```
public class Transaction
{
    public virtual void Validate()
    {
    }
}

public class Gift :
    Transaction
{
    public sealed override void Validate()
    {
        // final implementation in this branch
    }
}
```

Now a class derived from `Gift` cannot override `Validate` further.

sealed override Flow

EXAMPLE / FLOW
```
Transaction
    →
virtual Validate()

Gift
    →
sealed override Validate()
```

```
RecurringGift
 *  →
cannot override Validate()
```

INHERITANCE

Can a Derived Class Still Hide a Sealed Override?

It can potentially declare a separate member using:

EXAMPLE / FLOW

```
new
```

but that would be hiding, not participating in the virtual override chain. Microsoft documents this distinction.

Abstract Class Preview

Inheritance often works with:

EXAMPLE / FLOW

```
abstract
```

base classes.

Example:

C#

```
public abstract class Transaction
{
}
```

An abstract class cannot be directly instantiated:

C#

```
// new Transaction();
```

but can provide common members for derived classes.

Abstract classes will be covered properly in Topic 15.

Abstract Method Preview

Example:

C#

```
public abstract class Transaction
{
    public abstract decimal
        CalculateFee();
}
```

A concrete derived class must implement:

C#

```
public override decimal CalculateFee()
{
    ...
}
```

An abstract member has no normal implementation in the abstract class and is implicitly virtual.

Inheritance Hierarchy Example

EXAMPLE / FLOW

```
FinancialTransaction
        ↓
      Donation
        ↓
       Gift
        ↓
   RecurringGift
```

Each level should add meaningful specialization.

Avoid Deep Inheritance Trees

A hierarchy like:

EXAMPLE / FLOW

```
A
↓
B
↓
C
↓
D
↓
E
↓
F
↓
G
```

can become difficult to understand.

To understand `G`, you may need to inspect:

EXAMPLE / FLOW

```
F

E

D

C

B

A
```

This increases coupling and maintenance difficulty.

Prefer Shallow Hierarchies

Often better:

EXAMPLE / FLOW

```
Transaction
   ↓
Gift
```

or:

EXAMPLE / FLOW

```
Transaction
   ├─ Gift
   ├─ Refund
   └─ Adjustment
```

rather than many layers unless the domain genuinely requires them.

Upcasting

Because:

EXAMPLE / FLOW

```
Gift
IS A
Transaction
```

you can store a `Gift` reference in a `Transaction` variable.

Example:

C#

```
Gift gift = new Gift();

Transaction transaction = gift;
```

This conversion from derived to base is called:

EXAMPLE / FLOW

```
Upcasting
```

It is implicit.

Upcasting Flow

EXAMPLE / FLOW

```
Gift Object
    ↑
Gift Reference

Assign to
    ↓
Transaction Reference
    →
Same Gift Object
```

No second object is created.

Example

```
C#
Transaction transaction = new Gift();
```

The compile-time variable type is:

```
EXAMPLE / FLOW
Transaction
```

The runtime object type is:

```
EXAMPLE / FLOW
Gift
```

This distinction becomes critical in polymorphism.

What Members Can Base Reference See?

Suppose:

```
C#
Transaction transaction = new Gift();
```

Then you can directly access members known by:

```
EXAMPLE / FLOW
Transaction
```

You cannot directly access `Gift`-specific members unless you first establish/cast the runtime type.

Example:

```
C#
transaction.Amount;
```

may work.

But:

```
C#
// transaction.ConstituentId;
```

does not compile if `ConstituentId` exists only on `Gift`.

Compile-Time Type vs Runtime Type

Example:

```
C#
Transaction transaction = new Gift();
```

Compile-time type:

```
EXAMPLE / FLOW
Transaction
```

Runtime type:

EXAMPLE / FLOW

```
Gift
```

This distinction controls many behaviors involving:

EXAMPLE / FLOW

```
Member availability

Overload resolution

Virtual dispatch

Casting
```

Topic 14 will expand this heavily.

Downcasting

Suppose:

C#

```
Transaction transaction = new Gift();
```

You may convert back:

C#

```
Gift gift = (Gift)transaction;
```

This is:

EXAMPLE / FLOW

```
Downcasting
```

Unlike upcasting, it can fail if the runtime object is not actually a `Gift`.

Unsafe Downcast Example

C#

```
Transaction transaction = new RefundTransaction();

Gift gift = (Gift)transaction;
```

This can throw:

EXAMPLE / FLOW

```
InvalidCastException
```

because the runtime object isn't a `Gift`.

Prefer Pattern Matching

Instead of:

```C#
if (transaction is Gift)
{
    Gift gift = (Gift)transaction;
}
```

modern C# commonly uses:

```C#
if (transaction is Gift gift)
{
    Console.WriteLine(
        gift.ConstituentId);
}
```

The pattern:

EXAMPLE / FLOW

```
Checks runtime type
+
creates typed variable
```

Pattern matching will be covered deeply in Topic 26.

Inheritance and Object Equality

Two objects in an inheritance hierarchy may share state but still have different runtime types.

Example:

EXAMPLE / FLOW

```
Transaction

Gift : Transaction
```

When implementing custom equality, inheritance can make rules complicated.

Questions arise:

EXAMPLE / FLOW

```
Should a Gift equal a Transaction?

Should equality require same runtime type?

Should base properties participate?
```

This is one reason entity/value-object design should be deliberate.

Records and equality will be studied later.

Inheritance and Static Members

Static members belong to the declaring type rather than to a particular object.

They do not participate in runtime virtual dispatch the way instance virtual methods do.

Do not use static-member hiding as a substitute for polymorphism.

Example:

```C#
public class Base
{
    public static void Print()
    {
    }
}

public class Derived :
    Base
{
    public new static void Print()
    {
    }
}
```

Selection is based on the type used at the call site, not runtime object polymorphism.

Inheritance and Fields

Fields cannot be:

```
virtual
```

or:

```
override
```

in the same way methods/properties can.

If polymorphic state access is needed, expose it through:

```
Virtual Property

Virtual Method
```

rather than trying to override fields.

Bad Base Class Design

```C#
public class Transaction
{
    protected decimal Amount;

    protected string Status = string.Empty;

    protected List<string> AuditEntries = [];
}
```

Derived classes can directly manipulate every piece of internal state.

This produces:

```
Tight coupling
+
```

```
Broken invariants
+
Fragile hierarchy
```

Better Base Class Design

```C#
public class Transaction
{
    private decimal _amount;

    public decimal Amount
        =>
        _amount;

    protected void SetAmount(
        decimal amount)
    {
        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        _amount = amount;
    }
}
```

Derived classes use controlled extension points.

Protected Is Part of Your Extension API

Think of:

```
protected
```

members as an API for subclasses.

Once many subclasses depend on:

```
protected fields

protected methods

virtual methods
```

changing those members becomes harder.

So design them carefully.

Fragile Base Class Problem

Inheritance creates strong coupling between:

```
Base
```

and:

EXAMPLE / FLOW

```
Derived
```

A base-class implementation change can sometimes affect derived types unexpectedly.

Example:

EXAMPLE / FLOW

```
Base class changes
virtual-call sequence
    →
Derived override behavior changes
```

or:

EXAMPLE / FLOW

```
Base class introduces a new member
with same name as derived member
    →
hiding/versioning issues
```

This is sometimes called the:

EXAMPLE / FLOW

```
Fragile Base Class Problem
```

Example of Fragility

Version 1:

C#

```
public class Transaction
{
}
```

Derived application:

C#

```
public class Gift :
    Transaction
{
    public void Process()
    {
    }
}
```

Later base library adds:

C#

```
public class Transaction
{
    public void Process()
    {
    }
}
```

Now the derived `Process` hides the new base member.

The compiler may warn when rebuilt.

This is one of the versioning scenarios where C#'s `new` and `override` distinctions matter.

Inheritance vs Composition

This is one of the most important design decisions in OOP.

Inheritance:

```
EXAMPLE / FLOW
IS-A
```

Composition:

```
EXAMPLE / FLOW
HAS-A
```

Inheritance Example

```
EXAMPLE / FLOW
RecurringGift
IS A
Gift
```

Potentially reasonable.

Composition Example

```
EXAMPLE / FLOW
Gift
HAS A
PaymentProcessor
```

Do not write:

```
C#
public class Gift :
    PaymentProcessor
```

Instead:

```
C#
public class GiftService
{
    private readonly IPaymentProcessor
        _paymentProcessor;
}
```

This is composition.

Composition Often Creates Lower Coupling

Inheritance:

```
EXAMPLE / FLOW
Derived
    →
```

```
Strong knowledge
of base behavior
```

INHERITANCE

Composition:

EXAMPLE / FLOW

```
Object
    →
Depends on another component
through defined contract
```

Composition can often be easier to:

EXAMPLE / FLOW

```
Replace

Test

Mock

Change

Combine
```

This is why:

EXAMPLE / FLOW

```
Favor composition over inheritance
```

is a common design guideline.

It does not mean:

EXAMPLE / FLOW

```
Never use inheritance.
```

It means:

EXAMPLE / FLOW

```
Use inheritance when true subtype semantics exist.
```

Good Reason to Use Inheritance

Use it when:

EXAMPLE / FLOW

```
Derived type truly IS a base type

Base behavior is meaningful for derived objects

Derived objects can safely substitute for base objects

Common behavior belongs in base abstraction

Hierarchy is stable and understandable
```

Poor Reason to Use Inheritance

Do not use it merely because:

EXAMPLE / FLOW

```
Both classes have Amount property

You want access to one helper method

You want to avoid three lines of duplication

You need code from another class

You want global convenience
```

Composition or shared helper services may be more appropriate.

Liskov Substitution Preview

A critical SOLID principle later says, roughly:

> If `Gift` derives from `Transaction`, code expecting a `Transaction` should be able to use a `Gift` without surprising violations of the base contract.

This is the:

EXAMPLE / FLOW

```
Liskov Substitution Principle
```

which we will study deeply in Topic 31.

For now, inheritance implies more than code reuse.

It implies:

EXAMPLE / FLOW

```
substitutability
```

Bad Inheritance Example

Suppose:

C#

```
public class WritableDocument
{
    public virtual void Save()
    {
    }
}
```

Then:

C#

```
public class ReadOnlyDocument :
    WritableDocument
{
    public override void Save()
    {
        throw new NotSupportedException();
    }
}
```

This hierarchy is suspicious because:

EXAMPLE / FLOW

```
ReadOnlyDocument
```

cannot safely satisfy the contract expected of:

EXAMPLE / FLOW

```
WritableDocument
```

The type relationship may be wrong.

Fundraising Example: Good Hierarchy

Suppose your domain truly has:

EXAMPLE / FLOW

```
FinancialTransaction
    →
GiftTransaction

FinancialTransaction
    →
RefundTransaction
```

Common members:

EXAMPLE / FLOW

```
TransactionId
TenantId
Amount
CreatedAtUtc
```

Specialized behavior:

EXAMPLE / FLOW

```
Gift
    →
ConstituentId
CampaignId

Refund
    →
OriginalGiftId
RefundReason
```

This may be a reasonable hierarchy.

Complete Example

C#

```
public abstract class FinancialTransaction
{
    protected FinancialTransaction(
        long transactionId,
        long tenantId,
        decimal amount)
    {
        if (transactionId <= 0)
```

```
        {
            throw new ArgumentOutOfRangeException(
                nameof(transactionId));
        }

        if (tenantId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        TransactionId = transactionId;

        TenantId = tenantId;

        Amount = amount;

        CreatedAtUtc = DateTime.UtcNow;
    }

    public long TransactionId
    {
        get;
    }

    public long TenantId
    {
        get;
    }

    public decimal Amount
    {
        get;
        protected set;
    }

    public DateTime CreatedAtUtc
    {
        get;
    }

    public virtual string GetSummary()
    {
        return $"Transaction {TransactionId}: {Amount:C2}";
    }
}
```

Derived:

C#

```
public sealed class GiftTransaction :
    FinancialTransaction
{
    public GiftTransaction(
        long transactionId,
        long tenantId,
        long constituentId,
        decimal amount,
```

```
        long? campaignId)
    : base(
        transactionId,
        tenantId,
        amount)
{
    if (constituentId <= 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(constituentId));
    }

    ConstituentId = constituentId;

    CampaignId = campaignId;
}

public long ConstituentId
{
    get;
}

public long? CampaignId
{
    get;
}

public override string GetSummary()
{
    return
        $"{base.GetSummary()}, "
        +
        $"Constituent: {ConstituentId}";
}
}
```

Construction Flow for Example

EXAMPLE / FLOW

```
new GiftTransaction(...)
    →
GiftTransaction constructor selected
    →
base(...)
    →
FinancialTransaction validates
TransactionId
TenantId
Amount
    →
base state initialized
    →
GiftTransaction validates
ConstituentId
    →
derived state initialized
    →
GiftTransaction ready
```

Calling Overridden Summary

```C#
FinancialTransaction transaction =
    new GiftTransaction(
        transactionId: 1001,
        tenantId: 1,
        constituentId: 5001,
        amount: 1000m,
        campaignId: 2001);

Console.WriteLine(
    transaction.GetSummary());
```

Because:

EXAMPLE / FLOW

```
GetSummary
```

is virtual and overridden, the runtime uses:

EXAMPLE / FLOW

```
GiftTransaction.GetSummary()
```

even though the variable type is:

EXAMPLE / FLOW

```
FinancialTransaction
```

Fundraising Example: Inheritance May NOT Be Appropriate

Consider:

EXAMPLE / FLOW

```
Campaign

Fund
```

Both might contain:

EXAMPLE / FLOW

```
Name

Code

TenantId
```

Does that mean:

EXAMPLE / FLOW

```
Campaign IS A Fund?
```

No.

Does:

EXAMPLE / FLOW

```
Fund IS A Campaign?
```

No.

Therefore don't create:

C#

```
class Campaign :
    Fund
```

merely to reuse fields.

Use independent types or shared composition/value objects.

Another Bad Example

Suppose both:

EXAMPLE / FLOW

```
Constituent

AppUser
```

have:

EXAMPLE / FLOW

```
Name

Email
```

That does not automatically make:

EXAMPLE / FLOW

```
Constituent IS AN AppUser
```

Inheritance should follow domain meaning.

ASP.NET Core Inheritance Examples

ASP.NET Core itself uses inheritance in places such as controllers.

Example:

C#

```
public class GiftsController :
    ControllerBase
{
}
```

Here:

EXAMPLE / FLOW

```
GiftsController
IS A
ControllerBase
```

The derived controller receives framework behavior exposed by `ControllerBase`.

ControllerBase Example

```C#
[ApiController]
[Route("api/gifts")]
public sealed class GiftsController :
    ControllerBase
{
    [HttpGet("{id:long}")]
    public IActionResult Get(
        long id)
    {
        return Ok(
            new
            {
                GiftId =
                    id
            });
    }
}
```

Methods such as:

```EXAMPLE / FLOW
Ok()

BadRequest()

NotFound()
```

come from the controller inheritance hierarchy.

Custom Base Controllers

You can create:

```C#
public abstract class ApiControllerBase :
    ControllerBase
{
}
```

and derive:

```C#
public sealed class GiftsController :
    ApiControllerBase
{
}
```

But do not put every shared service/property into a giant base controller.

Inheritance can create hidden coupling.

Often middleware, filters, services, or composition are better.

Exception Inheritance Example

Custom exceptions derive from exception types.

Example:

```C#
public sealed class GiftNotFoundException :
    Exception
{
    public GiftNotFoundException(
        long giftId)
        : base(
            $"Gift {giftId} was not found.")
    {
    }
}
```

Relationship:

```EXAMPLE / FLOW
GiftNotFoundException
IS AN
Exception
```

This is a natural use of inheritance.

Attribute Inheritance Example

Custom attributes derive from:

```EXAMPLE / FLOW
System.Attribute
```

Example:

```C#
public sealed class AuditAttribute :
    Attribute
{
}
```

Again:

```EXAMPLE / FLOW
AuditAttribute
IS AN
Attribute
```

This is a true framework hierarchy.

Middleware Does Not Need Inheritance

ASP.NET Core middleware often uses composition/conventions rather than an inheritance hierarchy.

Don't assume every framework feature should be modeled through base classes.

Modern .NET frequently favors:

```EXAMPLE / FLOW
Interfaces

Composition

Dependency Injection
```

over deep inheritance trees.

Common Mistake — Using Inheritance Only for Code Reuse

Wrong reasoning:

EXAMPLE / FLOW

```
Class A has useful method
    →
Class B should inherit A
```

Ask instead:

EXAMPLE / FLOW

```
Is B truly an A?
```

If not, inject/use A as a dependency or extract shared functionality.

Common Mistake — Public/Protected Fields

Bad:

C#

```
public class BaseEntity
{
    protected long Id;

    protected string Status = string.Empty;
}
```

Derived classes can modify state arbitrarily.

Better:

C#

```
public long Id
{
    get;
}

protected void ChangeStatusCore(
    string status)
{
    // validation
}
```

Common Mistake — Making Everything virtual

Bad:

C#

```
public virtual void Method1()
{
}

public virtual void Method2()
{
}

public virtual void Method3()
```

```
{
}
```

without a clear extension requirement.

Every virtual member becomes part of the subclassing contract.

Only mark members virtual when derived customization is intentionally supported.

Common Mistake — Forgetting virtual

Base:

```C#
public void Process()
{
}
```

Derived:

```C#
public override void Process()
{
}
```

does not compile.

Base member must be overridable.

Common Mistake — Using new When You Intended override

Example:

```C#
public new void Process()
```

can produce reference-type-dependent behavior.

If you intend true runtime specialization, use:

```EXAMPLE / FLOW
virtual + override
```

instead.

Common Mistake — Ignoring Hiding Warning

If compiler tells you:

```EXAMPLE / FLOW
member hides inherited member
```

don't ignore it blindly.

Determine whether you intend:

```EXAMPLE / FLOW
override

new
```

```
or
different member name
```

Common Mistake — Calling base Automatically

This:

```C#
public override void Validate()
{
    base.Validate();
}
```

is not automatically required.

Call base behavior only if the derived contract needs it.

Sometimes an override completely replaces base behavior.

Common Mistake — Deep Hierarchy

Avoid:

```EXAMPLE / FLOW
Entity
 ↓
AuditableEntity
 ↓
TenantEntity
 ↓
TransactionEntity
 ↓
DonationEntity
 ↓
GiftEntity
 ↓
OnlineGiftEntity
```

unless each level genuinely represents a stable abstraction.

Deep hierarchies increase:

```EXAMPLE / FLOW
Coupling

Complexity

Hidden behavior

Testing difficulty
```

Common Mistake — Using Base Class as Dependency Bag

Example:

```C#
public abstract class BaseService
{
    protected DbContext DbContext;

    protected ILogger Logger;
```

```
    protected HttpClient HttpClient;

    protected IConfiguration Configuration;

    protected IMapper Mapper;
}
```

Then every service inherits it.

Problems:

EXAMPLE / FLOW

```
Hidden dependencies

Unused dependencies

Tight coupling

Difficult testing

Poor SRP
```

Prefer constructor injection into each service for the dependencies it actually needs.

Common Mistake — Inheritance for Shared Utility Methods

Instead of:

C#

```
class GiftService :
    UtilityBase
```

just to obtain:

EXAMPLE / FLOW

```
FormatDate()

NormalizeCurrency()

BuildReference()
```

consider:

EXAMPLE / FLOW

```
Static helper

Extension method

Injected service

Value object
```

depending on responsibility.

Common Mistake — Derived Class Violates Base Contract

If base says:

EXAMPLE / FLOW

```
Process()
always succeeds
for valid input
```

and derived class says:

EXAMPLE / FLOW

```
Process()
throws NotSupportedException
for every call
```

the hierarchy may violate substitutability.

This is often a sign inheritance is the wrong relationship.

Common Mistake — Assuming Private Members Don't Exist in Derived Objects

The base object's private state still forms part of the derived object's base portion.

Derived code simply cannot directly access those private members under normal accessibility rules.

Common Mistake — Assuming Constructors Are Inherited

They are not.

Each class defines its own construction contract.

Derived constructors can invoke base constructors with:

EXAMPLE / FLOW

```
base(...)
```

Common Mistake — Trying Multiple Class Inheritance

Invalid:

C#

```
public class Gift :
    Transaction,
    AuditableEntity
{
}
```

when both are classes.

Use:

EXAMPLE / FLOW

```
One Base Class
+
Interfaces
```

or composition.

Common Mistake — Assuming sealed Means Immutable

`sealed` means:

EXAMPLE / FLOW

```
Cannot derive from class
```

INHERITANCE

It does not mean:

EXAMPLE / FLOW

```
Object state cannot change
```

A sealed class can still be completely mutable.

Common Mistake — Assuming protected Means Public to Derived Objects Everywhere

There are nuanced rules for protected access.

A derived class doesn't get arbitrary access to protected members through unrelated base-class instances.

Microsoft specifically documents that protected access from derived code must occur through an appropriate derived-type instance.

Performance Considerations

Inheritance itself is not inherently "slow."

The important runtime distinction is usually between:

EXAMPLE / FLOW

```
Non-Virtual Call

Virtual Call
```

Virtual dispatch may require runtime method selection.

But in normal business applications the performance difference is usually tiny compared with:

EXAMPLE / FLOW

```
Database Calls

HTTP Calls

Serialization

File I/O
```

Do not avoid good polymorphic design for micro-optimization.

Sealed Optimization

Because a sealed class cannot have further derived implementations, the runtime may sometimes make additional optimizations.

Microsoft notes that sealed classes/members can enable some runtime optimizations.

But:

EXAMPLE / FLOW

```
performance
```

should usually not be the primary reason to seal ordinary business types.

Deep Hierarchy Performance Is Usually a Design Problem First

If a hierarchy is difficult:

```
EXAMPLE / FLOW
8 layers deep
```

the main concern is usually:

```
EXAMPLE / FLOW
Maintainability
```

rather than a few extra runtime instructions.

Design clarity matters more.

Memory Considerations

A derived object contains state needed for:

```
EXAMPLE / FLOW
Base portion
+
Derived portion
```

Example:

```
EXAMPLE / FLOW
FinancialTransaction state
+
Gift-specific state
```

The object remains one runtime object.

Inheritance does not create a separate full base object alongside the derived object.

Security Considerations

Inheritance can expose sensitive behavior through:

```
EXAMPLE / FLOW
protected
```

members.

Be careful when designing extensible base classes that hold:

```
EXAMPLE / FLOW
Security tokens

Credentials

Tenant state

Authorization state
```

Derived classes may gain access to protected information.

Do Not Expose Secrets Through protected

Avoid:

```C#
protected string ApiSecret;
```

if derived types don't truly need direct access.

Prefer narrow operations.

Example:

```C#
protected HttpRequestMessage
    CreateAuthenticatedRequest(...)
```

can be safer than exposing the raw credential itself.

Tenant Isolation Example

Suppose:

```C#
public abstract class TenantEntity
{
    public long TenantId
    {
        get;
        protected set;
    }
}
```

Any derived class can change:

```EXAMPLE / FLOW
TenantId
```

That may be dangerous.

Safer:

```C#
public long TenantId
{
    get;
}
```

and initialize it in the base constructor.

Now derived classes cannot casually move an entity across tenants.

Security Rule

Inheritance should not allow a derived class to bypass invariants that protect:

```EXAMPLE / FLOW
Authorization

Tenant boundaries
```

```
Financial values

Sensitive state
```

Keep security-sensitive state as restricted as possible.

Practical Exercise — Basic Inheritance

```C#
public class Transaction
{
    public decimal Amount
    {
        get;
        init;
    }
}

public class Gift :
    Transaction
{
    public long ConstituentId
    {
        get;
        init;
    }
}
```

Create:

```C#
Gift gift =
    new Gift
    {
        Amount = 1000m,

        ConstituentId =
            5001
    };
```

`Gift` gets both:

EXAMPLE / FLOW

```
Amount
+
ConstituentId
```

Practical Exercise — Constructor Inheritance

Given:

```C#
public class Transaction
{
    public Transaction(
        decimal amount)
    {
        Amount = amount;
    }
```

```
    public decimal Amount
    {
        get;
    }
}
```

Write derived constructor:

```C#
public class Gift :
    Transaction
{
    public Gift(
        decimal amount,
        long constituentId)
        : base(
            amount)
    {
        ConstituentId = constituentId;
    }

    public long ConstituentId
    {
        get;
    }
}
```

Practical Exercise — protected

```C#
public class Transaction
{
    protected decimal Amount
    {
        get;
        set;
    }
}
```

Can a normal unrelated class access:

```C#
transaction.Amount
```

Answer:

EXAMPLE / FLOW

No.

Can a derived class use `Amount` inside its allowed derived context?

EXAMPLE / FLOW

Yes.

Practical Exercise — virtual and override

Base:

```C#
public class Transaction
{
    public virtual string GetDescription()
    {
        return "Transaction";
    }
}
```

Derived:

```C#
public class Gift :
    Transaction
{
    public override string GetDescription()
    {
        return "Gift";
    }
}
```

Call:

```C#
Transaction value = new Gift();

Console.WriteLine(
    value.GetDescription());
```

Output:

EXAMPLE / FLOW

```
Gift
```

Practical Exercise — new

Base:

```C#
public class Transaction
{
    public void Print()
    {
        Console.WriteLine(
            "Transaction");
    }
}
```

Derived:

```C#
public class Gift :
    Transaction
{
    public new void Print()
    {
        Console.WriteLine(
            "Gift");
```

```
    }
}
```

What prints?

```C#
Transaction transaction = new Gift();

transaction.Print();
```

Answer:

EXAMPLE / FLOW

```
Transaction
```

Practical Exercise — sealed

```C#
public sealed class GiftReference
{
}
```

Can this compile?

```C#
public class SpecialReference :
    GiftReference
{
}
```

Answer:

EXAMPLE / FLOW

```
No.
```

Practical Exercise — sealed override

```C#
public class A
{
    public virtual void Process()
    {
    }
}

public class B :
    A
{
    public sealed override void Process()
    {
    }
}
```

Can `C : B` override `Process()`?

EXAMPLE / FLOW

```
No.
```

Practical Exercise — Is-A or Has-A?

Relationship:

```
EXAMPLE / FLOW
Gift
Campaign
```

Question:

```
EXAMPLE / FLOW
Gift IS A Campaign?
```

No.

Instead:

```
EXAMPLE / FLOW
Gift HAS A Campaign association.
```

Use composition/association.

Practical Exercise — Upcasting

```
C#
Gift gift = new Gift();

Transaction transaction = gift;
```

Is this allowed?

```
EXAMPLE / FLOW
Yes.
```

Why?

```
EXAMPLE / FLOW
Gift IS A Transaction.
```

Practical Exercise — Safe Type Check

```
C#
if (transaction is Gift gift)
{
    Console.WriteLine(
        gift.ConstituentId);
}
```

This:

```
EXAMPLE / FLOW
checks type
+
provides Gift variable
```

without unsafe blind downcasting.

Interview Questions

What is inheritance in C#?

Inheritance allows a class to derive from another class and reuse, extend, or specialize its members and behavior.

The inherited class is called the:

EXAMPLE / FLOW

```
base class
```

and the inheriting class is the:

EXAMPLE / FLOW

```
derived class
```

What is a base class?

A base class is the more general class whose members are inherited by a derived class.

Example:

EXAMPLE / FLOW

```
Transaction
    →
Base

Gift
    →
Derived
```

What is a derived class?

A class that inherits from another class.

Example:

C#

```
class Gift :
    Transaction
```

What is an "is-a" relationship?

It describes subtype semantics.

Example:

EXAMPLE / FLOW

```
Gift IS A FinancialTransaction
```

If that sentence does not make conceptual sense, inheritance may be the wrong design.

Does C# support multiple class inheritance?

No.

A class can have only one direct base class.

It can, however, implement multiple interfaces.

Is inheritance transitive?

Yes.

If:

```
EXAMPLE / FLOW
C inherits B

B inherits A
```

then:

```
EXAMPLE / FLOW
C also inherits from A indirectly.
```

What is an inheritance hierarchy?

A chain/tree of base and derived types.

Example:

```
EXAMPLE / FLOW
Object
    ↓
Transaction
    ↓
Gift
    ↓
RecurringGift
```

What is the ultimate base type of normal C# classes?

`System.Object`.

A class without an explicitly specified base class implicitly derives from `object`.

Are constructors inherited?

No.

Instance constructors, static constructors, and finalizers are not inherited.

How does a derived constructor call a base constructor?

Using:

```
C#
: base(...)
```

Example:

```
C#
public Gift(
    decimal amount)
    : base(
        amount)
{
}
```

Which constructor runs first: base or derived?

The selected base constructor completes before the derived constructor body.

What does base mean in C#?

It provides access to the direct base class implementation from an instance context.

Common uses:

EXAMPLE / FLOW

```
base(...)

base.Method()
```

Are private members inherited?

The C# language specification says members other than constructors/finalizers/static constructors are inherited regardless of accessibility, but private accessibility prevents normal derived-class source code from directly accessing them.

What is protected?

`protected` makes a member accessible inside the declaring class and derived classes, subject to protected-access rules.

Why use protected instead of public?

Use `protected` when a member is intended as an extension point for derived classes but should not be generally exposed to all callers.

What is a virtual method?

A method declared with:

C#

```
virtual
```

that permits a derived class to provide an override.

Are methods virtual by default?

No.

C# instance methods are non-virtual unless explicitly marked with a suitable modifier such as `virtual` or `abstract`.

What does override do?

`override` supplies a specialized implementation of an inherited virtual/abstract/override member.

Runtime dispatch can select that implementation based on the actual object type.

Can you override a non-virtual method?

No.

The base member must be virtual, abstract, or already overriding a virtual member, and it must not be sealed.

What is method hiding?

A derived type defines a new member with the same name/signature as an inherited member rather than overriding it.

The:

```
C#
new
```

modifier explicitly communicates this intention.

What is the difference between new and override?

`override`:

```
EXAMPLE / FLOW
participates in runtime polymorphism
```

`new`:

```
EXAMPLE / FLOW
hides the base member
```

For hiding, reference type can change which member is selected.

For overriding, runtime object type controls virtual dispatch.

What happens if a derived method hides a base method without new?

The compiler generally issues a warning and the behavior is hiding. Using `new` makes the intention explicit.

Can properties be virtual and overridden?

Yes.

Methods, properties, indexers, and events can participate in virtual override behavior.

What is a sealed class?

A class that cannot be inherited.

Example:

```
C#
public sealed class GiftReference
{
}
```

What is a sealed override?

An overridden virtual member that cannot be overridden further down the inheritance hierarchy.

Example:

```
C#
public sealed override void Process()
{
}
```

Can a sealed class contain mutable state?

Yes.

`sealed` controls inheritance.

It does not mean immutable.

What is an abstract class?

A class intended to serve as a base type and which cannot be instantiated directly.

An abstract class can contain implemented and abstract members.

What is an abstract method?

A member declared without an implementation in an abstract class.

A concrete derived class must provide an override.

An abstract method is implicitly virtual.

What is upcasting?

Converting a derived reference to a base reference.

Example:

```C#
Gift gift = new Gift();

Transaction transaction = gift;
```

This is implicit and does not create another object.

What is downcasting?

Converting a base reference to a more derived type.

Example:

```C#
Gift gift = (Gift)transaction;
```

It can fail at runtime if the object is not actually the requested derived type.

What is safer than blind downcasting?

Pattern matching:

```C#
if (transaction is Gift gift)
{
}
```

This checks the runtime type before using the derived reference.

What is the difference between compile-time type and runtime type?

Example:

```C#
Transaction transaction = new Gift();
```

Compile-time type:

EXAMPLE / FLOW

```
Transaction
```

Runtime type:

EXAMPLE / FLOW

```
Gift
```

The difference is central to polymorphism.

Is inheritance only for code reuse?

No.

True inheritance should model:

EXAMPLE / FLOW

```
subtyping
+
is-a relationship
+
substitutability
```

Code reuse by itself is often better achieved through composition.

What is composition over inheritance?

Instead of inheriting merely to obtain functionality, an object receives or owns another object that provides that functionality.

Example:

EXAMPLE / FLOW

```
GiftService
HAS AN
IPaymentProcessor
```

rather than:

EXAMPLE / FLOW

```
GiftService
IS A
PaymentProcessor
```

What is the fragile base-class problem?

Derived types can depend strongly on implementation details and extension points of a base class.

Changes to the base class may unexpectedly affect derived classes.

Deep inheritance or poorly designed virtual/protected APIs increase this risk.

When should inheritance be used?

When:

EXAMPLE / FLOW

```
Derived truly is a specialized base type
```

```
Base contract applies to derived type
Substitution makes sense
Shared behavior belongs naturally in base
Hierarchy is stable
```

INHERITANCE

When should inheritance not be used?

Avoid it when:

EXAMPLE / FLOW

```
You only want helper methods
Relationship is has-a
Derived would violate base contract
Hierarchy becomes deep
Composition offers cleaner flexibility
```

Why shouldn't every method be virtual?

Each virtual member becomes an extension point and subclass contract.

That increases compatibility and maintenance obligations.

Only expose virtual behavior intentionally.

Why can protected fields be dangerous?

They let derived types directly manipulate base state and potentially violate invariants.

Protected methods or properties with validation can often provide safer extension points.

Does inheritance automatically improve design?

No.

Incorrect inheritance can create more coupling and complexity than duplicated code.

The relationship must make semantic sense.

Best Practices

Use inheritance when there is a genuine:

EXAMPLE / FLOW

```
IS-A
```

relationship.

Keep hierarchies shallow when possible.

Prefer:

EXAMPLE / FLOW

```
private
```

for base implementation details.

Use:

EXAMPLE / FLOW

```
protected
```

only for intentional subclass extension points.

Avoid protected mutable fields where derived classes can bypass invariants.

Use:

EXAMPLE / FLOW

```
virtual
```

only when derived types are intentionally allowed to change behavior.

Use:

EXAMPLE / FLOW

```
override
```

for true specialization of virtual behavior.

Use:

EXAMPLE / FLOW

```
new
```

only when member hiding is genuinely intended.

Don't ignore member-hiding compiler warnings.

Use:

EXAMPLE / FLOW

```
base(...)
```

to initialize required base state.

Use:

EXAMPLE / FLOW

```
base.Method()
```

when the derived behavior intentionally includes base behavior.

Remember constructors are not inherited.

Remember private members may exist in the inherited object state even when derived source cannot access them directly.

Use `sealed` when further inheritance should be prohibited.

Use `sealed override` when a particular virtual behavior should stop being overridable.

Do not use inheritance only to reuse a utility method.

Prefer composition for:

EXAMPLE / FLOW

```
HAS-A
```

relationships.

Avoid giant base classes containing unrelated dependencies.

Do not use inheritance to hide dependency injection.

Keep security-sensitive state private rather than broadly protected.

Ensure derived classes preserve the expectations of the base-class contract.

Quick Revision

Concept	Meaning
Inheritance	Deriving one class from another
Base class	General parent type
Derived class	Specialized child type
`:`	Inheritance syntax
`is-a`	Main conceptual inheritance relationship
Single inheritance	One direct class base
Transitive inheritance	Derived indirectly from ancestor classes
`object`	Ultimate base type for normal classes
`protected`	Accessible to declaring/derived classes
`base(...)`	Call base constructor
`base.Member`	Access base implementation
`virtual`	Member may be overridden
`override`	Specialized virtual implementation
`new`	Hide inherited member
`sealed class`	Cannot be derived from
`sealed override`	Cannot be overridden further
Abstract class	Non-instantiable base abstraction
Upcast	Derived → base reference
Downcast	Base → derived reference
Hierarchy	Chain/tree of inherited types
Composition	Has-a relationship
Substitutability	Derived should safely act as base

Complete Fundraising Example

C#

```
public abstract class FinancialTransaction
{
    protected FinancialTransaction(
        long transactionId,
        long tenantId,
        decimal amount,
        DateTime createdAtUtc)
    {
```

```
        if (transactionId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(transactionId));
        }

        if (tenantId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        TransactionId = transactionId;

        TenantId = tenantId;

        Amount = amount;

        CreatedAtUtc = createdAtUtc;
    }

    public long TransactionId
    {
        get;
    }

    public long TenantId
    {
        get;
    }

    public decimal Amount
    {
        get;
        private set;
    }

    public DateTime CreatedAtUtc
    {
        get;
    }

    public virtual string GetSummary()
    {
        return
            $"Transaction {TransactionId}, "
            +
            $"Amount: {Amount:C2}";
    }

    protected void ChangeAmountCore(
        decimal amount)
    {
        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }
```

```
        Amount = amount;
    }
}
```

Derived `GiftTransaction`:

C#

```
public sealed class GiftTransaction :
    FinancialTransaction
{
    public GiftTransaction(
        long transactionId,
        long tenantId,
        long constituentId,
        decimal amount,
        long? campaignId,
        DateTime createdAtUtc)
        : base(
            transactionId,
            tenantId,
            amount,
            createdAtUtc)
    {
        if (constituentId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(constituentId));
        }

        ConstituentId = constituentId;

        CampaignId = campaignId;
    }

    public long ConstituentId
    {
        get;
    }

    public long? CampaignId
    {
        get;
        private set;
    }

    public void ChangeAmount(
        decimal amount)
    {
        ChangeAmountCore(
            amount);
    }

    public void ChangeCampaign(
        long? campaignId)
    {
        CampaignId = campaignId;
    }

    public override string GetSummary()
    {
        return
            $"{base.GetSummary()}, "
            +
            $"Constituent: {ConstituentId}, "
```

```
            +
            $"Campaign: {CampaignId?.ToString() ?? "None"}";
    }
}
```

Another derived transaction:

C#

```
public sealed class RefundTransaction :
    FinancialTransaction
{
    public RefundTransaction(
        long transactionId,
        long tenantId,
        long originalGiftId,
        decimal amount,
        string reason,
        DateTime createdAtUtc)
        : base(
            transactionId,
            tenantId,
            amount,
            createdAtUtc)
    {
        if (originalGiftId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(originalGiftId));
        }

        ArgumentException.ThrowIfNullOrWhiteSpace(
            reason);

        OriginalGiftId = originalGiftId;

        Reason = reason;
    }

    public long OriginalGiftId
    {
        get;
    }

    public string Reason
    {
        get;
    }

    public override string GetSummary()
    {
        return
            $"{base.GetSummary()}, "
            +
            $"Refund for Gift: {OriginalGiftId}, "
            +
            $"Reason: {Reason}";
    }
}
```

Usage:

```C#
FinancialTransaction first =
    new GiftTransaction(
        transactionId: 1001,
        tenantId: 1,
        constituentId: 5001,
        amount: 1000m,
        campaignId: 2001,
        createdAtUtc: DateTime.UtcNow);

FinancialTransaction second =
    new RefundTransaction(
        transactionId: 1002,
        tenantId: 1,
        originalGiftId: 1001,
        amount: 100m,
        reason: "Partial refund",
        createdAtUtc: DateTime.UtcNow);

Console.WriteLine(
    first.GetSummary());

Console.WriteLine(
    second.GetSummary());
```

Conceptually:

```EXAMPLE / FLOW
FinancialTransaction Reference
            ↓
  ┌─────────┴─────────┐
  ↓                   ↓
GiftTransaction   RefundTransaction
```

Each object shares the base contract:

```EXAMPLE / FLOW
TransactionId
TenantId
Amount
CreatedAtUtc
GetSummary()
```

but provides specialized state and behavior.

Full Inheritance Flow

```EXAMPLE / FLOW
Need New Type
    →
Does it represent
a specialization of existing type?
    →
No
    →
Prefer composition / separate type

Yes
    →
Does "Derived IS A Base"
make sense?
```

```
    →
No
    →
Don't inherit

Yes
    →
Inheritance may be appropriate
```

Then:

EXAMPLE / FLOW

```
Base Class
    →
Common State

Common Behavior

Protected Extension Points

Virtual Members Where Intentional
```

Derived:

EXAMPLE / FLOW

```
Derived Class
    →
Inherited Members
    +
Specialized State
    +
Specialized Behavior
```

Construction:

EXAMPLE / FLOW

```
new Derived()
    →
Base Constructor
    →
Derived Constructor
    →
Object Ready
```

Customization:

EXAMPLE / FLOW

```
Base virtual method
    →
Derived override
    →
Runtime dispatch
```

Hiding:

EXAMPLE / FLOW

```
Base non-virtual method
    →
Derived new method
    →
Reference-type-dependent selection
```

Stop further derivation:

EXAMPLE / FLOW

```
sealed class
```

or:

EXAMPLE / FLOW

```
sealed override
```

Final Inheritance Decision Model

EXAMPLE / FLOW

```
Question 1:
Is B really an A?

        ↓

No
    →
Use composition

Yes
    →
Continue
```

EXAMPLE / FLOW

```
Question 2:
Can B safely be used
where A is expected?

        ↓

No
    →
Hierarchy is probably wrong

Yes
    →
Continue
```

EXAMPLE / FLOW

```
Question 3:
Does A expose only intentional
extension points?

        ↓

No
    →
Redesign base class

Yes
    →
Inheritance may be suitable
```

EXAMPLE / FLOW

```
Question 4:
Should behavior vary by derived type?
```

```
*       ↓

Yes
    →
virtual / override
```

EXAMPLE / FLOW

```
Question 5:
Should further derivation stop?

        ↓

Yes
    →
sealed
```

Final End-to-End Concept Flow

EXAMPLE / FLOW

```
Base Class
    →
General Contract
    →
Common State
    →
Common Behavior
```

EXAMPLE / FLOW

```
Derived Class
    →
IS A Base Class
    →
Inherits Common Members
    →
Adds Specialized Members
```

Construction:

EXAMPLE / FLOW

```
Derived Object Requested
    →
Base Constructor
    →
Derived Constructor
    →
One Complete Derived Object
```

Behavior:

EXAMPLE / FLOW

```
Normal Base Member
    →
Inherited / Reused
```

EXAMPLE / FLOW

```
Virtual Base Member
    →
Derived override
```

```
    →
Runtime chooses most-derived implementation
```

EXAMPLE / FLOW

```
Hidden Member
    →
new
    →
Not true polymorphic override
```

Design:

EXAMPLE / FLOW

```
IS-A
    →
Inheritance

HAS-A
    →
Composition
```

The most important lesson from this topic is:

> **Inheritance models specialization, not merely code reuse. A derived class should genuinely be a kind of its base class and should preserve the base class's contract. Use `virtual` and `override` for intentional runtime specialization, `new` only for deliberate member hiding, `sealed` to stop inheritance when appropriate, and composition when the relationship is actually "has-a" rather than "is-a."**

For the Fundraising Platform, inheritance can be useful when concepts genuinely form a hierarchy such as:

EXAMPLE / FLOW

```
FinancialTransaction
    →
GiftTransaction

FinancialTransaction
    →
RefundTransaction
```

but it would usually be incorrect simply to make:

EXAMPLE / FLOW

```
Gift
inherit Campaign

Campaign
inherit Fund

Constituent
inherit AppUser
```

because shared fields do not automatically imply subtype relationships.

Next roadmap topic: Topic 14 — Polymorphism.